Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448: Midterm Project

Team Members: Salman Asif, Chris Li, Surya Harikrishnan

Date: October 10, 2026

1. Introduction and Problem Definition

Large Language Models (LLMs) like ChatGPT and Claude have become increasingly popular for generating written content. As these models continue to improve, it has become harder to tell which AI model produced a particular response. However, different LLMs may have their own writing styles, word choices, and patterns that can help distinguish them from one another.

The goal of this project is to see whether machine learning can identify which LLM generated a response based on the text it produces. To test this, we implemented two deep learning models: a Convolutional Neural Network (CNN) and a Recurrent Neural Network (RNN), specifically a Bidirectional Long Short-Term Memory (BiLSTM) model.

Our project focuses on three research questions, numbered as in the assignment (RQ3, cross-domain generalization, was not investigated):

> **RQ1:** Can we identify which LLM generated a response by looking only at its output?
>
> **RQ2:** Does including the user's original prompt help us identify the LLM more accurately?
>
> **RQ4:** What characteristics distinguish different LLMs?

To answer these questions, we trained and tested both models using different types of text input. We compared their performance using accuracy and macro-F1 scores to see how well each model could identify the correct LLM. We also looked at how the results changed depending on whether the models received the prompt, the generated response, or both. For RQ4, we used simple style features and a TF-IDF logistic regression model to examine which characteristics distinguish the LLMs. Finally, we discuss what these results tell us and some limitations of our approach.

2. Dataset and Preprocessing

2.1 Dataset Description

We use the publicly available **TXD-22: A Large-Scale Benchmark Dataset**, obtained from Kaggle.

Dataset source: https://www.kaggle.com/datasets/mdsadiqiqbal/txd-22-a-large-scale-benchmark-dataset

The original dataset contains three primary columns:

- **Question:** The prompt provided to an LLM.
- **Answer:** The response generated by the LLM.
- **Source:** The LLM responsible for generating the response.

For our experiments, these columns were standardized to **`input, output, and label`**, respectively.

Our processed dataset contains six LLM classes. This allows us to formulate the task as a six-class classification problem in which each response is assigned to its source model.

2.2 Dataset Splitting

The processed dataset contains 18,000 examples, divided into three subsets:

Table 1. Dataset split

Dataset Split	Number of Examples
Training	14,448
Validation	1,794
Testing	1,758
Total	**18,000**

The training set is used to learn model parameters, the validation set is used to select the best-performing training epoch, and the test set is reserved for final evaluation.

2.3 Text Preprocessing

Before training our models, we first had to prepare the text so it could be processed properly. We used a tokenizer that breaks the text into smaller pieces, such as words, punctuation marks, and newline characters. We then created a vocabulary using the training data, allowing up to 30,000 tokens. Words or tokens had to appear at least twice to be included. We also used special tokens to handle unknown words and fill in shorter sequences. To keep the input sizes consistent, we set a maximum length of 256 tokens. Text that was longer than this limit was shortened, while shorter text was padded to reach the same length.

For our second research question (RQ2), we tested three different ways of providing text to the models:

1. **Input only:** We used just the original prompt given to the LLM.
2. **Output only:** We used only the response generated by the LLM.

3. **Input + Output:** We combined the original prompt and the generated response, separating them with a special marker.

For each of these three approaches, we created a separate vocabulary using the corresponding training data. This allowed us to compare how well the models performed depending on the information they were given.

3. Model Architectures and Training

3.1 Convolutional Neural Network (CNN)

Our CNN implementation uses a TextCNN-style architecture designed for text classification. The model begins with an embedding layer that converts token IDs into 128-dimensional dense vectors. These embeddings are passed through parallel one-dimensional convolutional layers with kernel sizes of 3, 4, and 5. Each convolutional layer uses 100 filters, allowing the model to identify local textual patterns of different lengths. After convolution, ReLU activation and global max pooling are applied. The resulting feature representations are concatenated and passed through a dropout layer with a dropout probability of 0.5. Finally, a fully connected classification layer produces scores for the six LLM classes. This architecture is designed to capture local linguistic patterns that may distinguish the writing styles of different LLMs.

3.2 Recurrent Neural Network (BiLSTM)

Our second model uses a Bidirectional Long Short-Term Memory network. The model begins with a 128-dimensional embedding layer. The embedded sequences are processed by a bidirectional LSTM with a hidden size of 128. Unlike a standard unidirectional recurrent network, a BiLSTM processes text in both forward and backward directions. This allows it to capture contextual information from both preceding and following tokens. The model uses packed sequences to account for padding and applies dropout with a probability of 0.3 before classification. A final linear layer maps the learned representation to six output classes.

3.3 Training Configuration

Both models use the following general training configuration:

Table 2. Training configuration

Parameter	Setting
Framework	PyTorch
Batch size	64
Maximum sequence length	256 tokens
Embedding dimension	128
Optimizer	Adam
Learning rate	0.001
Loss function	Cross-entropy
Random seed	42
CNN training epochs	10
RNN training epochs	20

During training, validation accuracy is measured after each epoch. The model state achieving the highest validation accuracy is selected for final test evaluation. We evaluate classification performance using accuracy and macro-averaged F1 score. Macro-F1 assigns equal importance to each class when averaging performance.

4. Experimental Results

4.1 RQ1: Can We Identify the Source LLM from Its Response?

Our first research question investigates whether the generated response alone contains enough information to identify the source LLM. We trained both the CNN and BiLSTM using only the **`output`** column.

The results are shown below.

Table 3. RQ1 test results (output only)

Model	Test Accuracy	Macro-F1
CNN	81.40%	81.35%
BiLSTM	75.71%	75.92%

Both deep learning models achieved substantially higher accuracy than the approximately 16.67% chance-level baseline for six equally likely classes. The CNN achieved the highest accuracy among the two neural architectures, outperforming the BiLSTM by approximately 5.69 percentage points. These results suggest that LLM-generated responses contain recognizable textual patterns that can be learned by neural classifiers. The CNN may benefit from detecting local combinations of words and punctuation, while the BiLSTM captures sequential relationships throughout the text. However, the CNN and BiLSTM experiments used different numbers of training epochs, so the comparison should not be interpreted as a fully controlled architecture comparison.

4.2 RQ2: Does the User's Prompt Help Identify the LLM?

Our second research question investigates whether the user's original prompt provides additional information about the source LLM. We evaluate three input representations: input only, output only, and input combined with output.

CNN Results

Table 4. CNN results by input representation

Input Representation	Accuracy	Macro-F1
Input only	17.35%	10.25%
Output only	81.40%	81.35%
Input + Output	80.15%	80.28%

BiLSTM Results

Table 5. BiLSTM results by input representation

Input Representation	Accuracy	Macro-F1
Input only	17.29%	12.46%
Output only	75.71%	75.92%
Input + Output	76.85%	77.00%

4.3 Interpretation of RQ2

The input-only experiments produced accuracy values of approximately 17% for both architectures. Because the task involves six classes, this is close to the chance-level accuracy of 16.67% under a balanced-class assumption. This suggests that the user's prompt alone contains limited information about which LLM generated the response. In contrast, output-only classification achieved substantially higher accuracy. The CNN reached 81.40%, while the

BiLSTM reached 75.71%. This indicates that the generated responses contain stronger source-identifying signals than the prompts.

Interestingly, combining the input and output did not give a consistent improvement. The CNN decreased by about 1.25 points (81.40% to 80.15%), while the BiLSTM increased by about 1.14 points (75.71% to 76.85%). Because the changes are small and point in opposite directions, and because results vary by a few points between runs, we do not treat them as meaningful. One possible explanation for the lack of improvement is that the prompt adds little useful information for distinguishing source models. The fixed maximum sequence length of 256 tokens may also play a role, since concatenated prompts and responses can cause some response content to be truncated. Further experiments would be needed to determine which explanation is responsible for the observed performance differences.

4.4 RQ4: What Characteristics Distinguish Different LLMs?

Classification accuracy shows that the LLMs can be told apart, but not why. To investigate which characteristics matter, we analyzed the output text of the training set and ran additional experiments on the same train and test splits. Because these analyses are fast to run, we used simple style features and the TF-IDF logistic regression baseline instead of the neural networks. They therefore show which signals exist in the data, but not exactly which signals the CNN and BiLSTM use.

Table 6 shows the average style features of each LLM's responses in the training set.

Table 6. Average style features per LLM (output text, training set)

LLM	Avg. chars	Avg. words	Punct. marks	Vocab. variety	Avg. word length
ChatGPT	1,723	241	44.1	0.66	6.17
Claude	2,271	296	43.4	0.65	6.65
DeepSeek	1,955	260	65.4	0.72	6.52
Gemini	1,871	263	53.1	0.66	6.08
Llama	1,404	208	32.5	0.57	5.77
Qwen	1,646	217	45.0	0.73	6.61

The LLMs differ in measurable ways. Claude writes the longest responses (about 2,271 characters on average) and Llama the shortest (about 1,404). DeepSeek uses the most punctuation and Llama the least, and Llama also has the lowest vocabulary variety (the share of unique words). Formatting symbols are almost absent: responses average at most 0.06 line breaks and 0.62 markdown symbols, and bullet lists are virtually absent, so the responses in this dataset are essentially plain text.

To test how much each kind of signal contributes, we removed or isolated signals one at a time, as shown in Table 7.

Table 7. Signal ablations on output text (test set)

Setting	Accuracy	Macro-F1
Words only (baseline)	86.29%	86.28%
Words + punctuation + newlines	88.85%	88.85%
Formatting stripped (no markdown, no newlines)	88.45%	88.45%
Truncated to first 50 words	58.36%	57.43%
Punctuation and newlines only (no words)	55.46%	55.08%
Style features only (random forest)	63.77%	63.72%
Response length only (random forest)	37.71%	37.68%

A classifier using only the style features reached 63.77% accuracy, well above the 16.67% chance level, so structure and style carry real signal. Response length alone reached only 37.71%, and punctuation and newlines alone reached 55.46%. In the random forest, the most important features were vocabulary variety, length, and punctuation count. Words are still the strongest signal: the word-only model reached 86.29%, and adding punctuation and newlines as tokens raised it to 88.85%.

Removing signals gave mixed results. Stripping markdown symbols and line breaks lowered accuracy by only 0.40 points (88.85% to 88.45%), which fits the near absence of formatting in this dataset. Truncating every response to its first 50 words caused the largest drop, to 58.36%. This change removes both length differences and the later content of each response, so we cannot tell how much of the drop comes from length alone. Since length by itself is a weak signal, much of the drop probably reflects the model having less text to learn from.

Finally, the words most strongly associated with each LLM by the baseline model suggest differences in phrasing habits (Table 8). For example, Qwen favors example-giving phrases such as "for example" and "for instance", and Llama favors "such as" and "used to". These are associations learned by a linear model, so they describe patterns in this dataset and do not show that an LLM always writes this way.

Table 8. Examples of words and phrases with high weight for each LLM

LLM	Example words and phrases
ChatGPT	especially, helps, in summary
Claude	through, including, creating
DeepSeek	like, while, via
Gemini	often, an, are
Llama	such as, used to, range of
Qwen	for example, fostering, addressing

5. Analysis and Discussion

5.1 CNN Versus BiLSTM

The CNN achieved higher test accuracy than the BiLSTM for output-only and combined input, and the two models were tied near chance level for input-only. For output-only classification, the CNN achieved 81.40% accuracy compared with 75.71% for the BiLSTM. This suggests that local textual patterns may be particularly informative for the LLM source-attribution task. CNNs are designed to identify local features through convolutional filters, whereas BiLSTMs model relationships across sequences. The results are consistent with the possibility that differences in local wording, punctuation, and phrasing contribute to source identification. However, accuracy alone does not establish exactly which features each model learned.

5.2 Why Input-Only Classification Performs Poorly

Input-only classification produced accuracy near chance level for both models. This result is reasonable because the user's prompt generally describes a task or question rather than the identity of the LLM that will answer it. If similar prompts are provided to different LLMs, the prompt itself may not contain meaningful information about which model generated the eventual response. The low input-only Macro-F1 scores also indicate that classification performance was weak across the six classes.

5.3 Comparison with a Traditional Machine Learning Baseline

In addition to the neural architectures, our project includes a TF-IDF-based logistic regression baseline. The previously recorded baseline achieved approximately 86.30% accuracy and

86.30% Macro-F1, outperforming both neural models in the reported output-only experiments. This is an important finding because more complex neural architectures do not necessarily outperform traditional machine learning methods on every text classification task. TF-IDF features can be highly effective when word-frequency patterns provide strong information about class membership. However, differences in experimental settings should be checked before drawing definitive conclusions about model superiority.

5.4 Limitations

Our experiments have several limitations.

- First, all input sequences are limited to 256 tokens. This can remove potentially informative content from longer responses, especially when prompts and outputs are combined.
- Second, the CNN and BiLSTM were trained for different numbers of epochs. Consequently, the results do not isolate architectural differences under identical training budgets.
- Third, our dataset does not include explicit task-domain labels. Therefore, we did not conduct a cross-domain generalization experiment to determine whether the learned patterns transfer across different task categories.
- Fourth, the experiments focus on identifying LLM sources represented in the dataset. We have not established how well these classifiers would generalize to previously unseen LLMs or substantially different datasets.
- Fifth, each result comes from a single training run. The BiLSTM output-only accuracy differed by about 3 percentage points between two runs (78.67% and 75.71%), and repeated CNN output-only runs ranged from about 80.4% to 81.7% (the 81.40% reported here is the result saved in the repository), so differences of a few points between models or input settings should be interpreted cautiously. The final BiLSTM numbers come from a seeded script (`experiments/bilstm_runs.py`) run on a single Colab GPU.
- Finally, our feature analysis (RQ4) used simple style features and a TF-IDF model rather than the neural networks, so it shows which signals exist in the data but not exactly which ones the CNN and BiLSTM rely on.

6. Lessons Learned and Experience

This project provided practical experience in developing and evaluating deep learning models for natural language processing. One important lesson was that data preprocessing plays a major role in model performance. Tokenization, vocabulary construction, padding, and truncation all affect the information available to the classifier. We also learned how different neural architectures approach text classification. The CNN uses convolutional filters to identify local patterns, while the BiLSTM processes sequences in both directions to learn contextual representations. Implementing both architectures helped us understand their structural differences and how they can be applied to the same classification problem.

Another important lesson came from comparing different input representations. Initially, we considered whether including both the prompt and response would improve classification. However, the experiments showed that adding the prompt gave no consistent improvement over output-only classification. This demonstrated the importance of testing assumptions experimentally rather than assuming that providing more information will always improve model performance.

We also gained experience organizing a collaborative machine learning project using Git and GitHub. Separate development branches and pull requests allowed us to work on different model implementations and experiments before merging changes into the main repository. Finally, the project reinforced the importance of interpreting results carefully. Differences in training settings, sequence lengths, and dataset characteristics can affect conclusions, and strong classification accuracy does not automatically explain which features a model uses.

7. Conclusion

In this project, we investigated whether machine learning classifiers can identify the source LLM from generated responses. We implemented and evaluated two required neural architectures: a TextCNN and a BiLSTM. For output-only classification, the CNN achieved 81.40% accuracy and the BiLSTM achieved 75.71%, demonstrating that generated responses contain useful information for source attribution. Our second research question showed that the original prompts alone provide little predictive information. Both models achieved approximately 17% accuracy when trained using input-only text. Combining prompts and responses also did not consistently improve performance over output-only classification. Overall, our findings suggest that source-specific information is more strongly represented in LLM-generated responses than in the prompts that produce them. Our feature analysis (RQ4) showed that vocabulary carries most of the signal, while style features such as vocabulary variety, length, and punctuation are also informative on their own. Future work could investigate cross-domain generalization, which features the neural models rely on, longer input sequences, and the ability to recognize previously unseen LLMs.

References

[1] TXD-22: A Large-Scale Benchmark Dataset. Kaggle. https://www.kaggle.com/datasets/mdsadiqiqbal/txd-22-a-large-scale-benchmark-dataset